Zhi-Hua Zhou

Machine Learning

Translated by Shaowu Liu

The slides of this chapter
are translated by Yu-Xuan Huang

Springer

Chapter 15
Rule Learning

Outline

- Basic Concepts
- Sequential Covering
- Pruning Optimization
- First-order Rule Learning
- Hints to Inductive Logic Programming

Basic Concepts

- Rules in machine learning:

  If ……, then …….

  - Regression

    $\text{If } \hat{h}(\mathbf{x}) = \hat{y} \qquad y = \hat{y} \text{ then}$

  - Classification

    $\text{If } h(\mathbf{x}) > 0 \qquad class = 1 \text{ then} \quad \text{If } h(\mathbf{x}) \leq 0 \qquad ; \qquad class = -1 \text{ then}$

  - Clustering

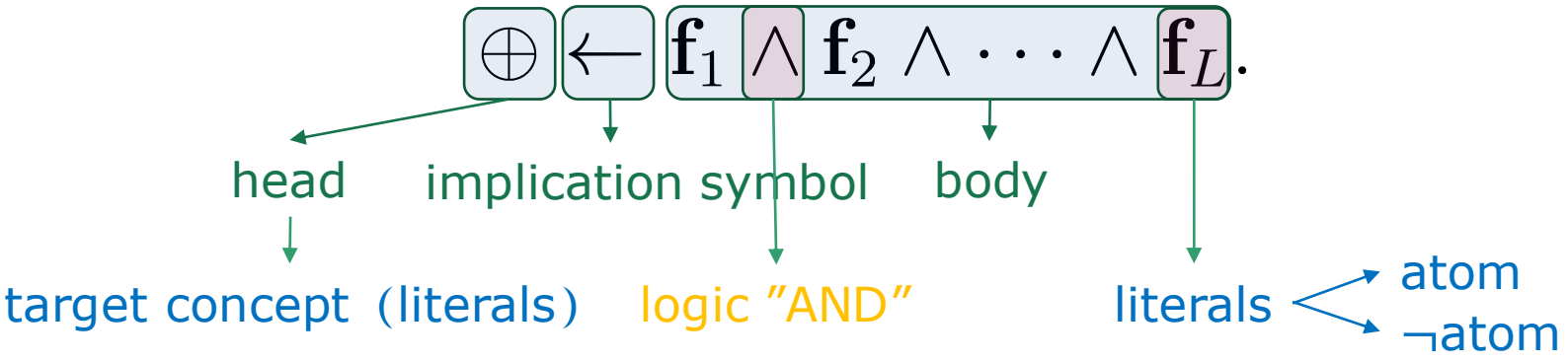
    $\text{If } \forall j \neq i, dist(\mathbf{x}, c_i) \leq dist(\mathbf{x}, c_j) \qquad C(\mathbf{x}) = c_i \qquad \text{, then}$

  - ……

- Rule Learning: A branch of machine learning based on symbolic rules

Basic Concepts

- Logic rules



Pronounced as: if (literal1 and literal2 and…), then (target) holds

- Rule set

$$\text{Rule 1: ripe} \leftarrow (\text{root} = \text{curly}) \wedge (\text{umbilicus} = \text{hollow})$$
$$\text{Rule 2: } \neg\text{ripe} \leftarrow (\text{texture} = \text{blurry}).$$

  - Conflict resolution: voting, ordering, and meta-rule methods

Basic Concepts

- Propositional logic $\longrightarrow$ Propositional rule
  - Propositional atoms: $A, B, C, \ldots$
  - Logical connectors: $\leftarrow, \rightarrow, \leftrightarrow, \wedge, \vee, \neg, \ldots$

$$R \leftarrow P \wedge Q.$$

$$\text{ripe} \leftarrow (\text{root} = \text{curly}) \wedge (\text{umbilicus} = \text{hollow})$$

- First-order logic $\longrightarrow$ First-order rule
  - Constant: $a, b, c, \ldots, 1, 2, 3, \ldots$
  - Variable: $A, B, C, \ldots$
  - (arity n) predicate/function: $p/n, f/n, \ldots$
  - Term: $constant|variable|function(term_1, term_2, \ldots)$
  - Atomic formula: $predicate(term_1, term_2, \ldots)$

$$father(X, Y), N(39), Even(\sigma(1))$$

  - Logical connective: $\leftarrow, \rightarrow, \leftrightarrow, \wedge, \vee, \neg, \ldots$
  - Quantifier: $\exists, \forall$

$$\forall X(p(f(X)) \leftarrow p(X)).$$

$$\forall X(N(\sigma(X)) \leftarrow N(X))$$

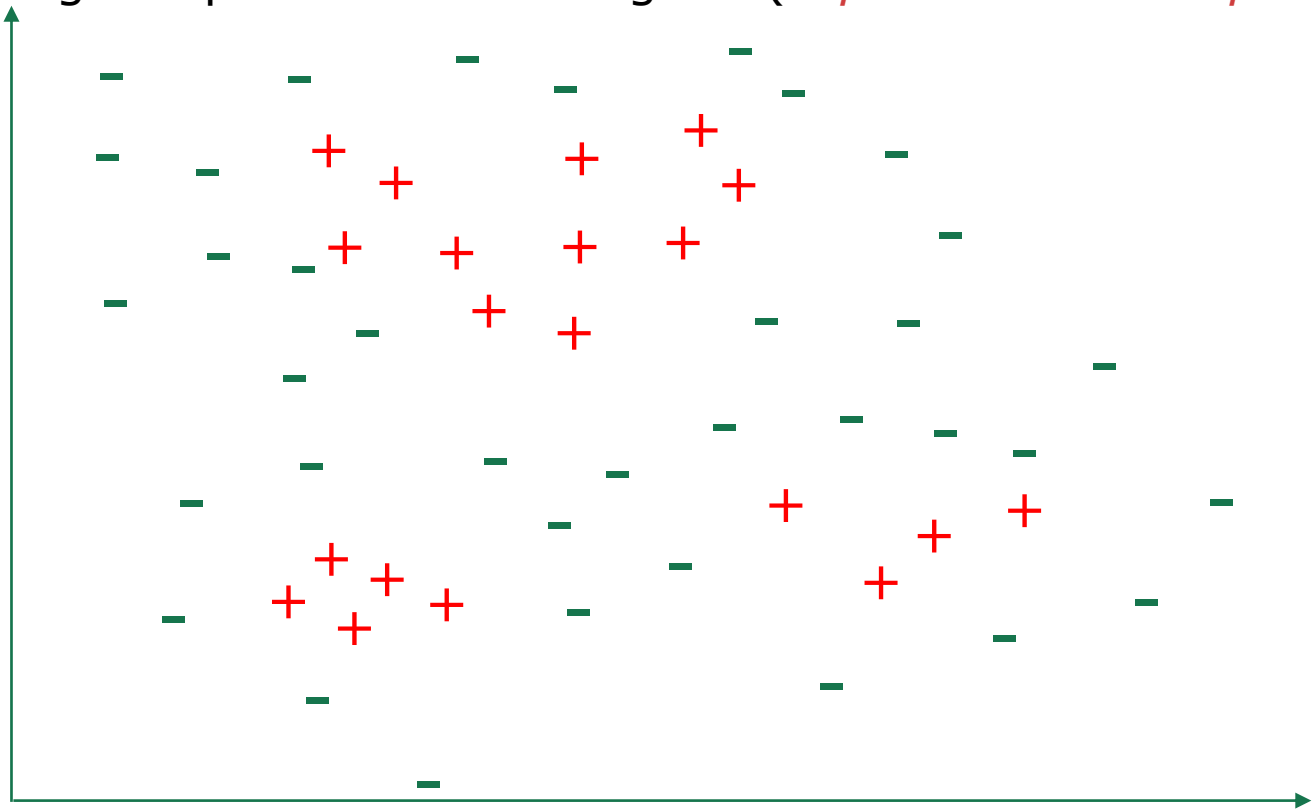
$\sigma(X)$ is the successor of X

Outline

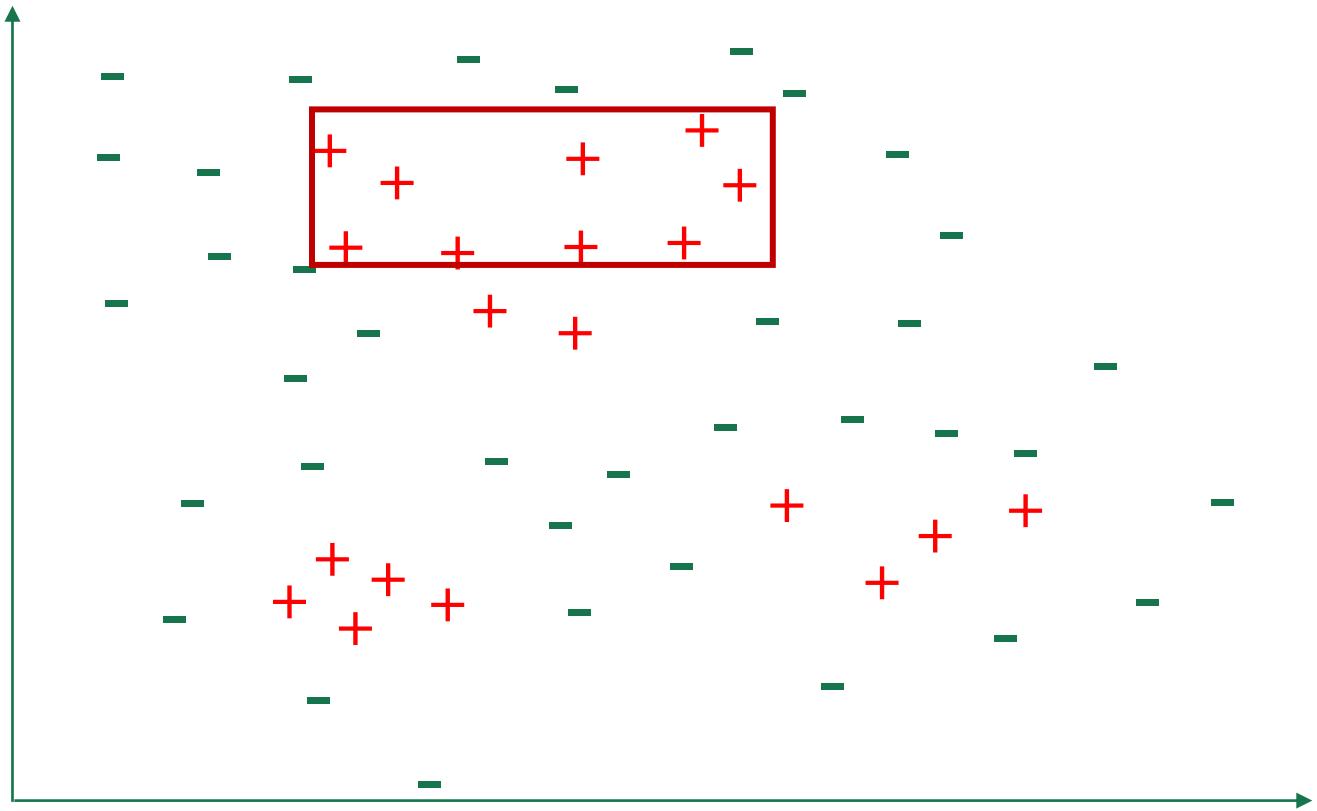
- Basic Concepts
- Sequential Covering
- Pruning Optimization
- First-order Rule Learning
- Inductive Logic Programming

Sequential Covering

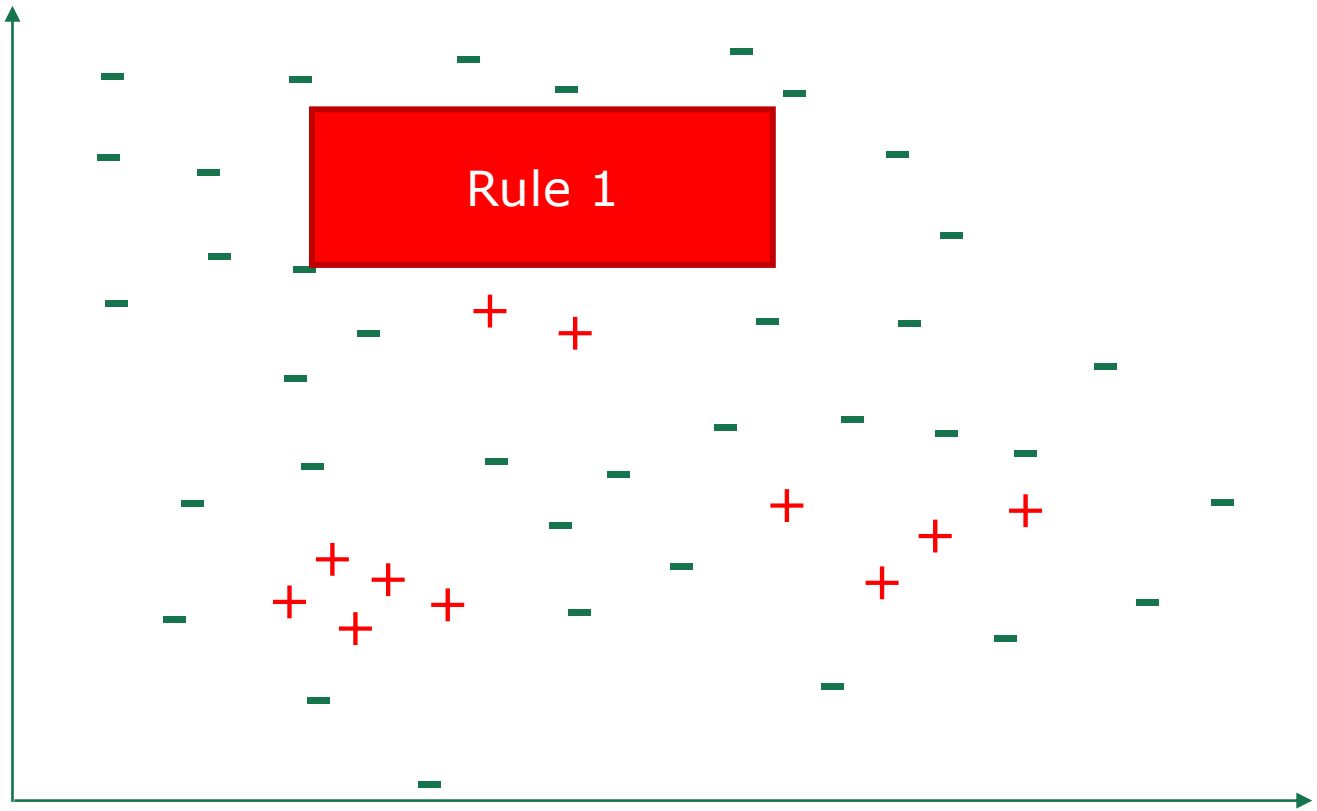
- Every time a rule is learned, remove all samples covered by it from the training set, and the learning process repeats with the remaining samples in the training set (*separate-and-conquer*).



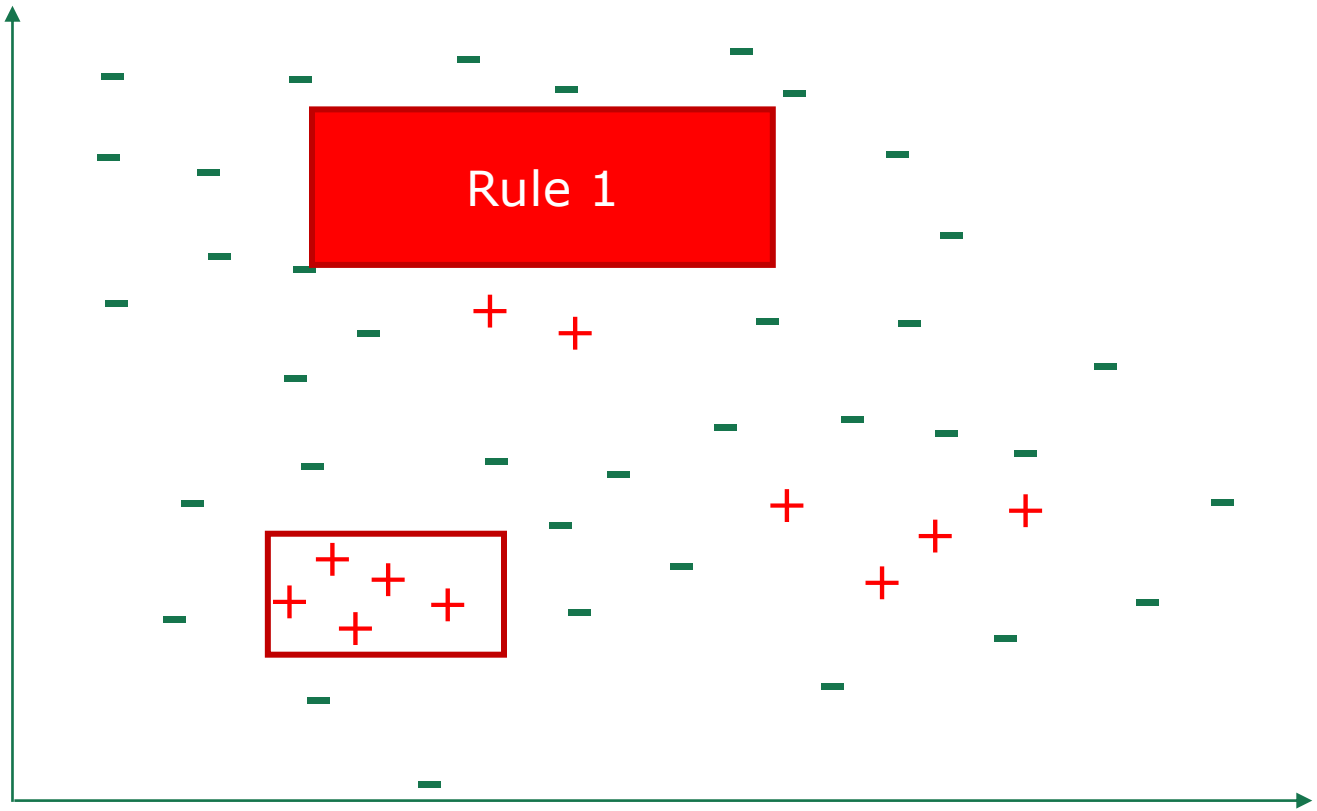
Sequential Covering



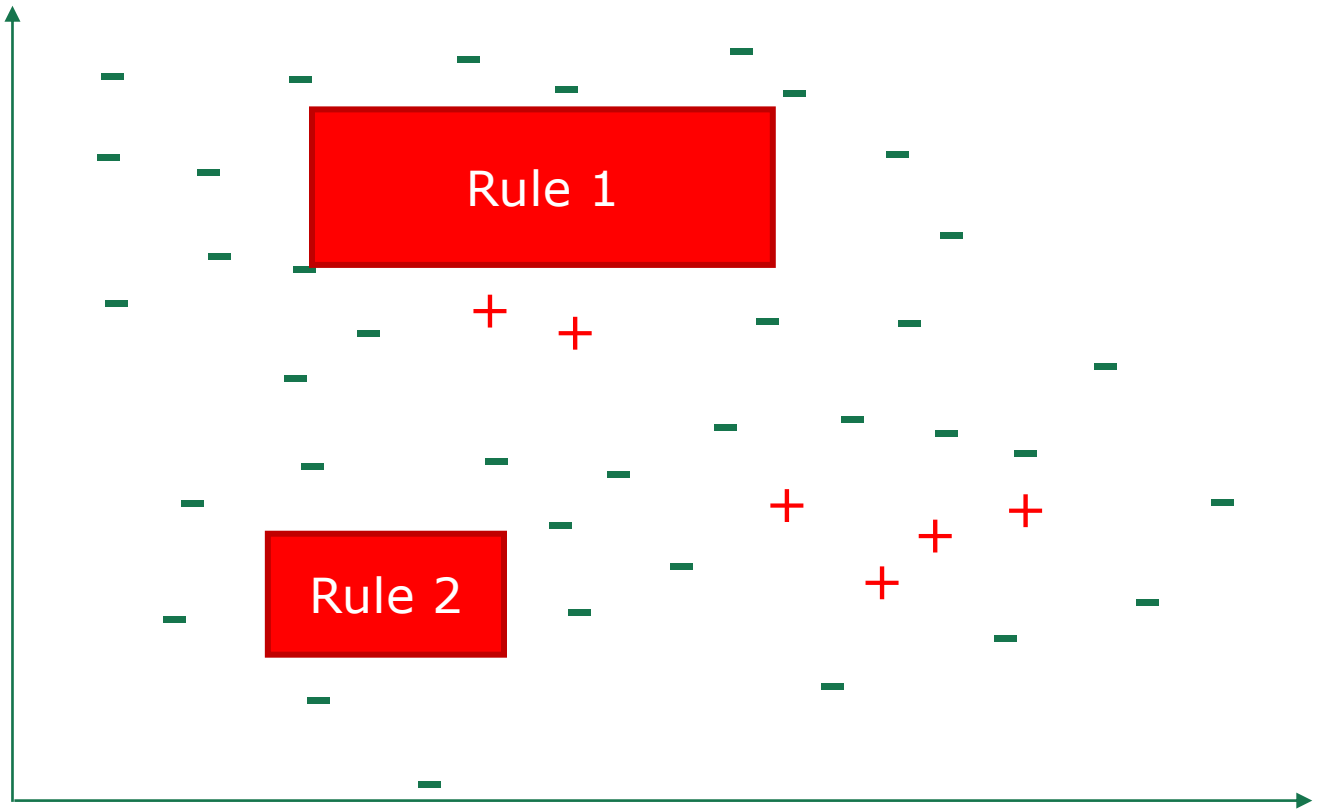
Sequential Covering



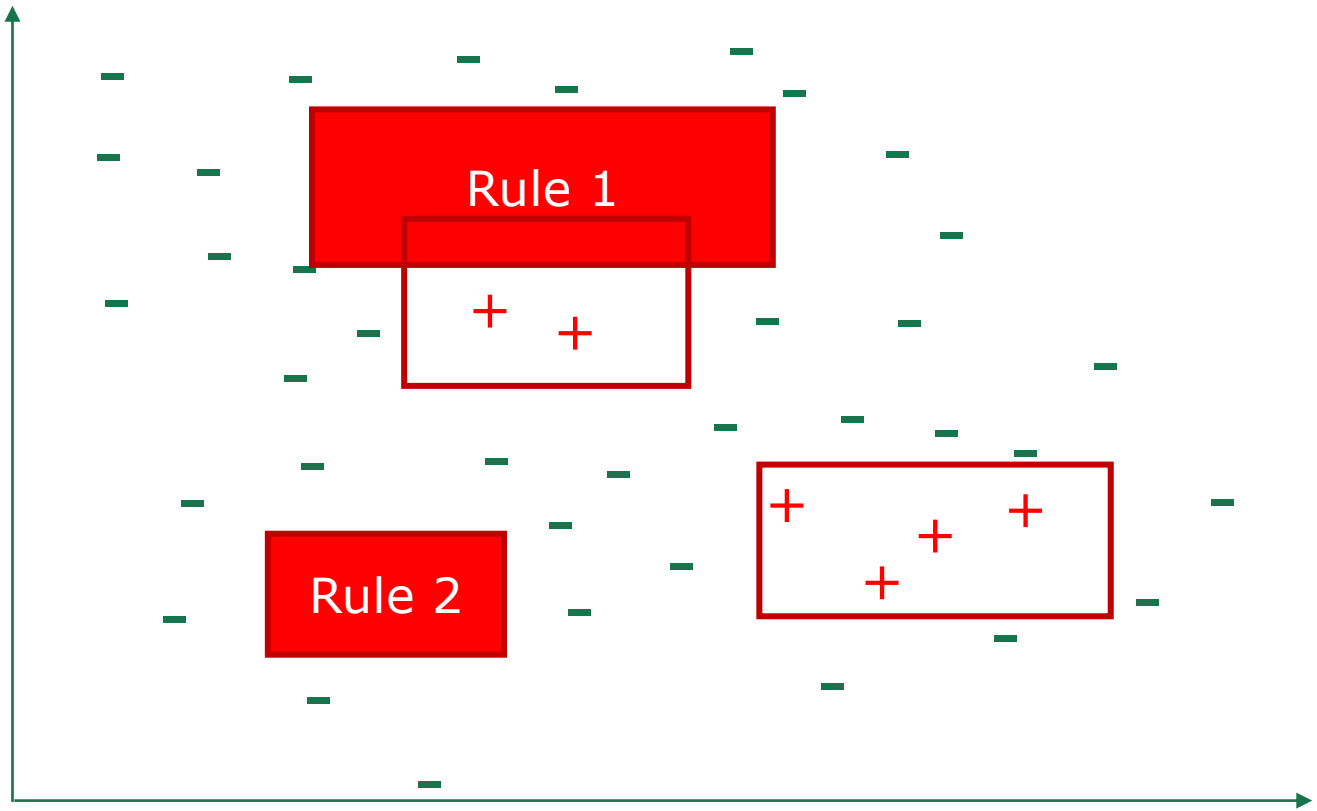
Sequential Covering



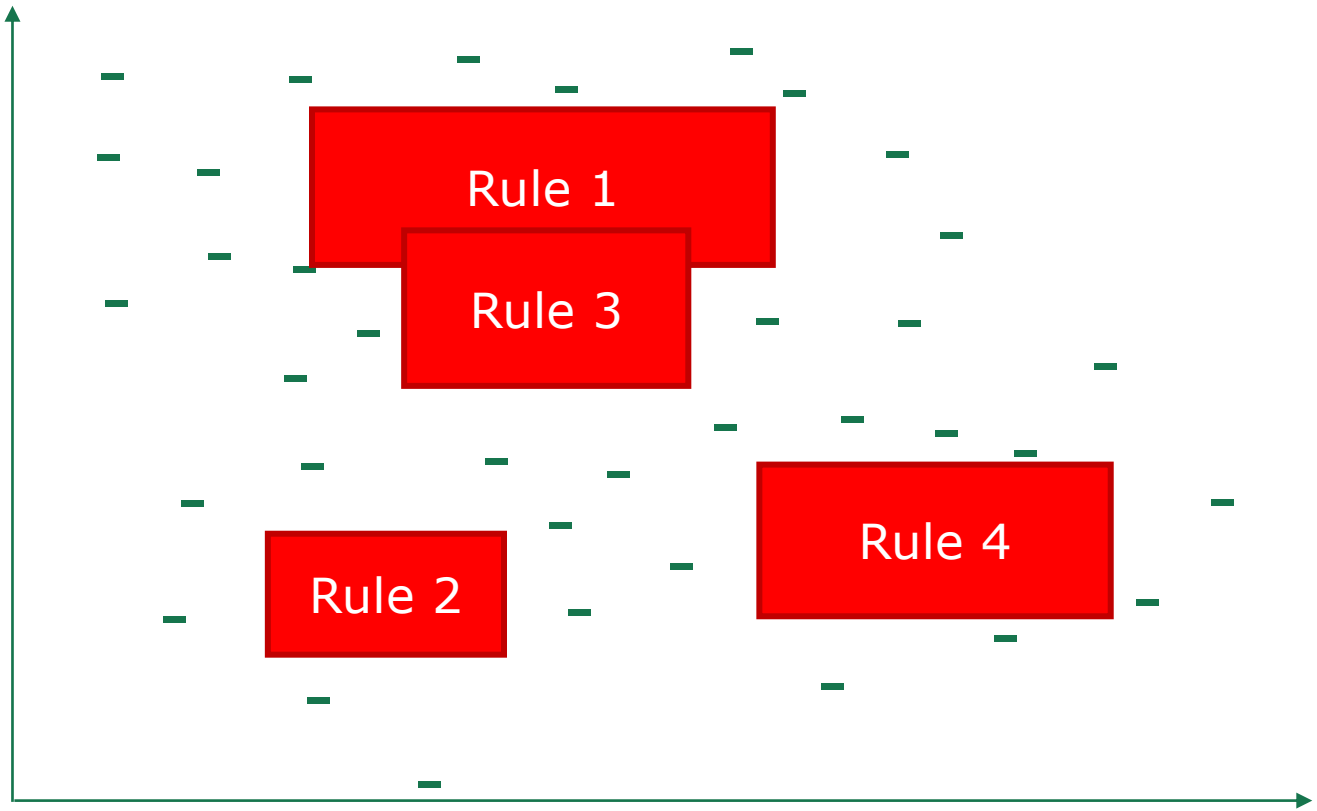
Sequential Covering



Sequential Covering



Sequential Covering



Outline

- Basic Concepts
- Sequential Covering
  - Single rule learning
- Pruning Optimization
- First-order Rule Learning
- Inductive Logic Programming

Single rule learning

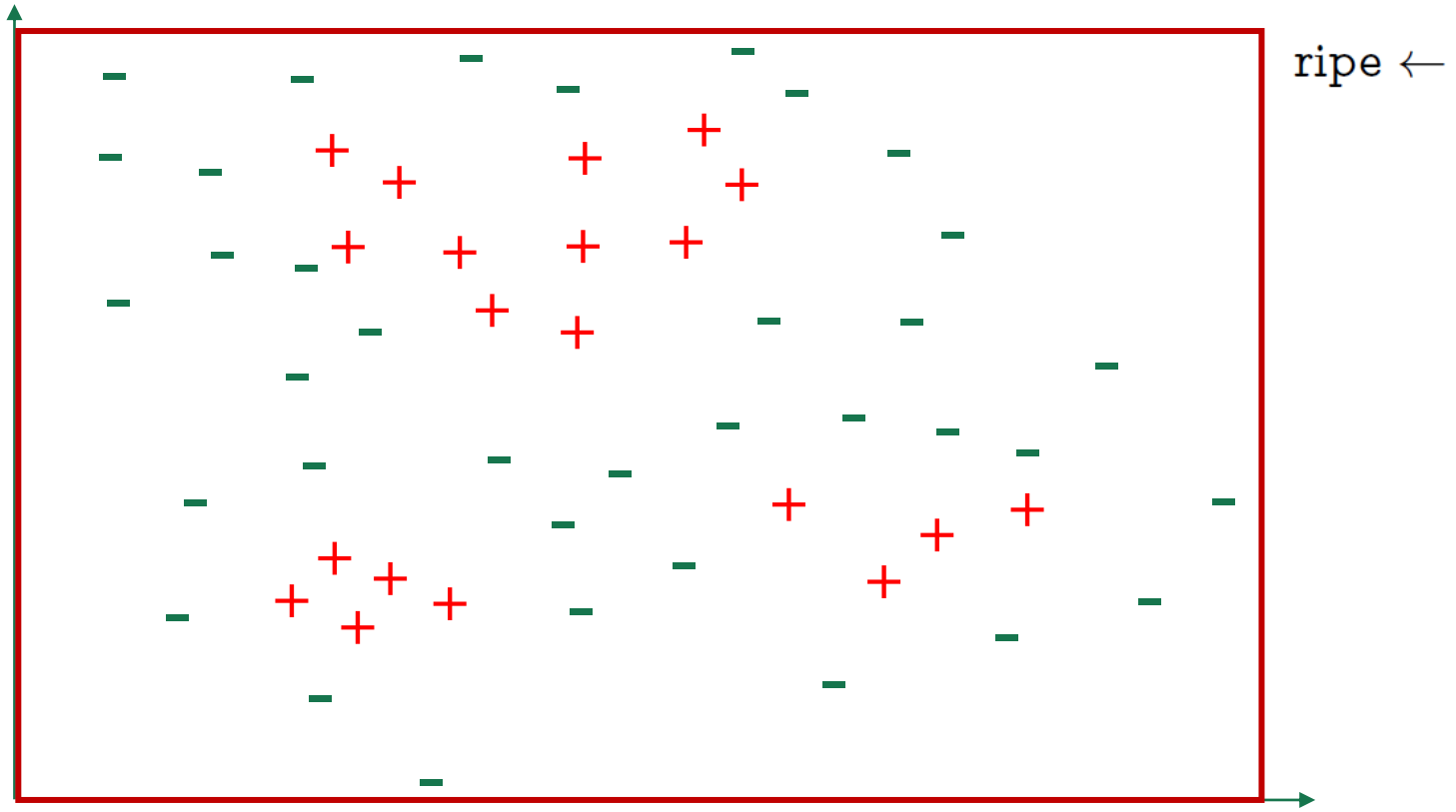
- Target: find a group of optimal literals to form the rule body

- Essence: search problem
  - Large search space, easy to cause combination explosion

- Strategy：
  - Top-down: general to special (specialization)
  - Bottom-up: special to general (generalization)

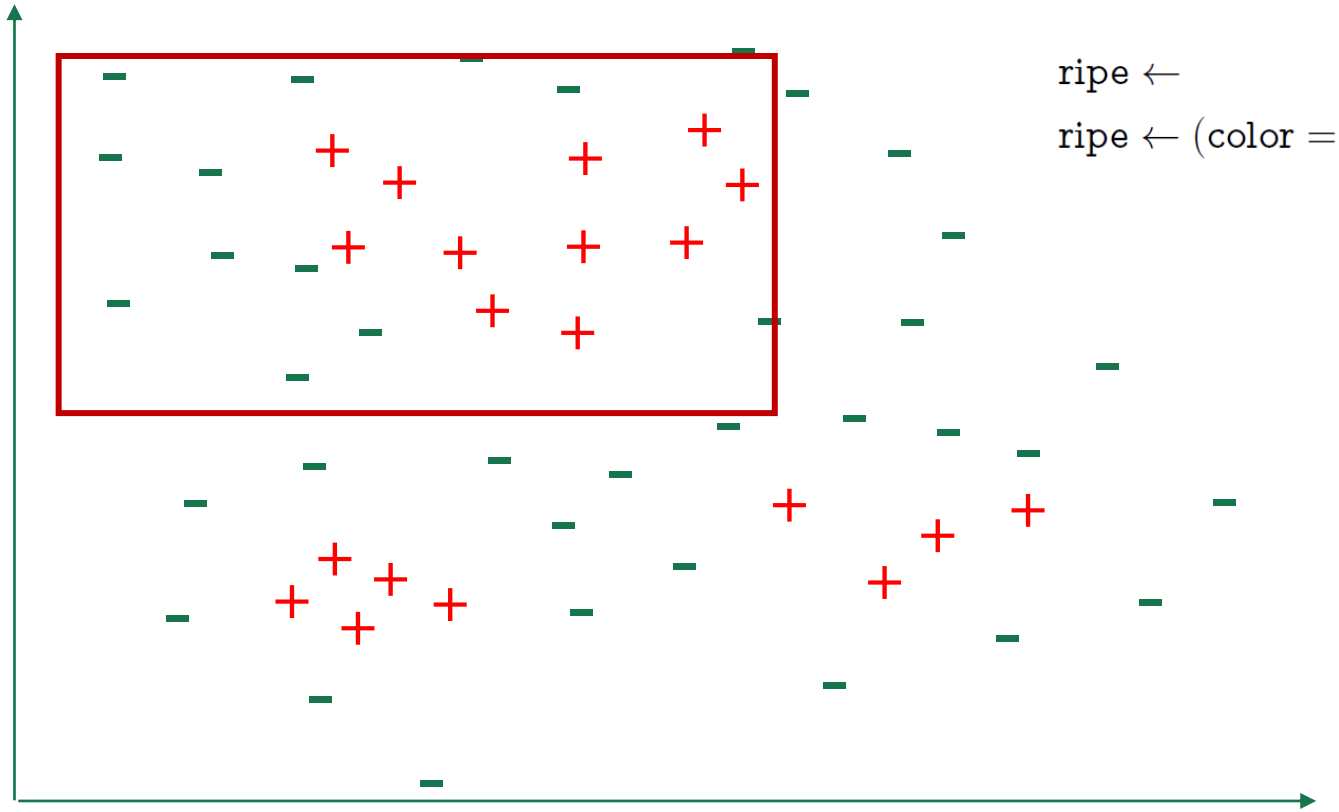
Single rule learning

- Top-down strategy: general to special (specialization)



Single rule learning

- Top-down strategy: general to special (specialization)

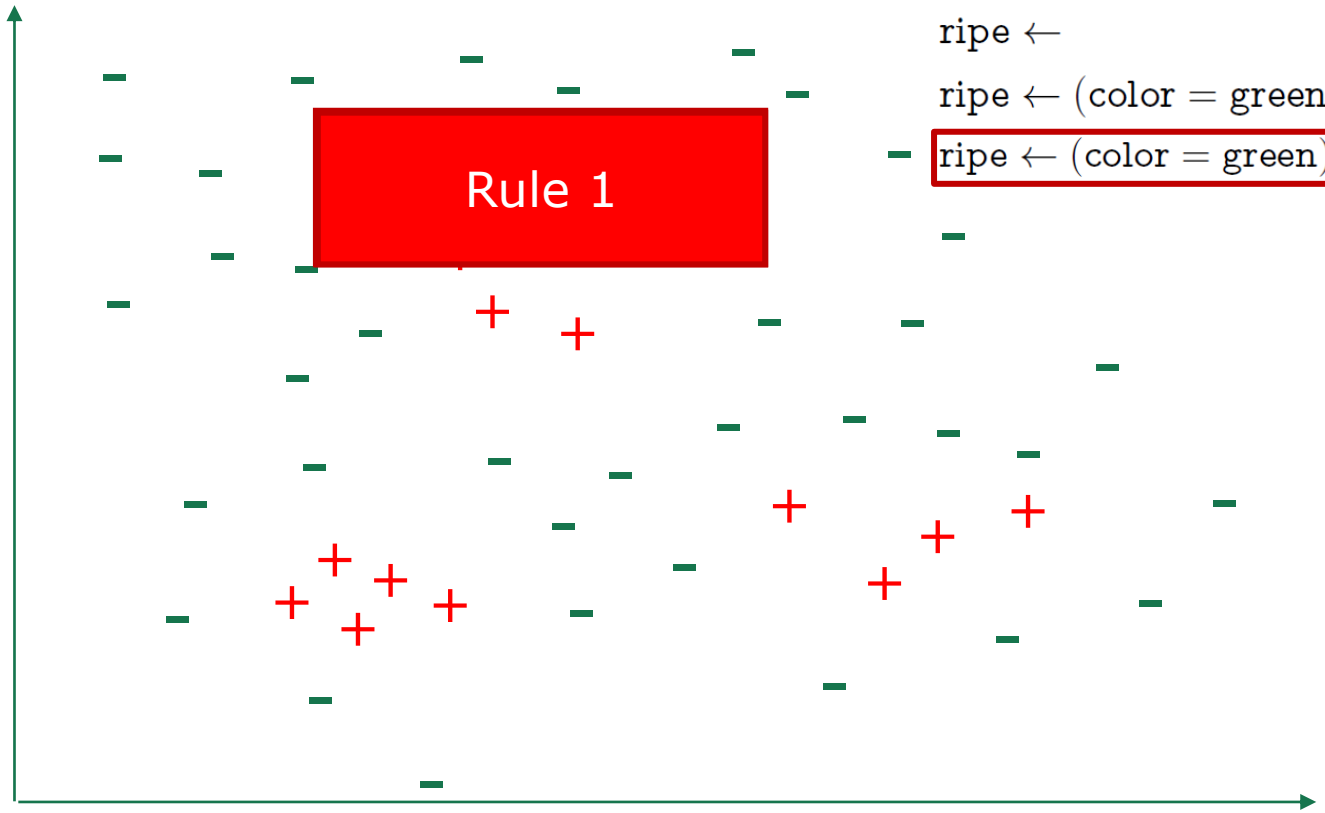


ripe ←

ripe ← (color = green).

Single rule learning

- Top-down strategy: general to special (specialization)



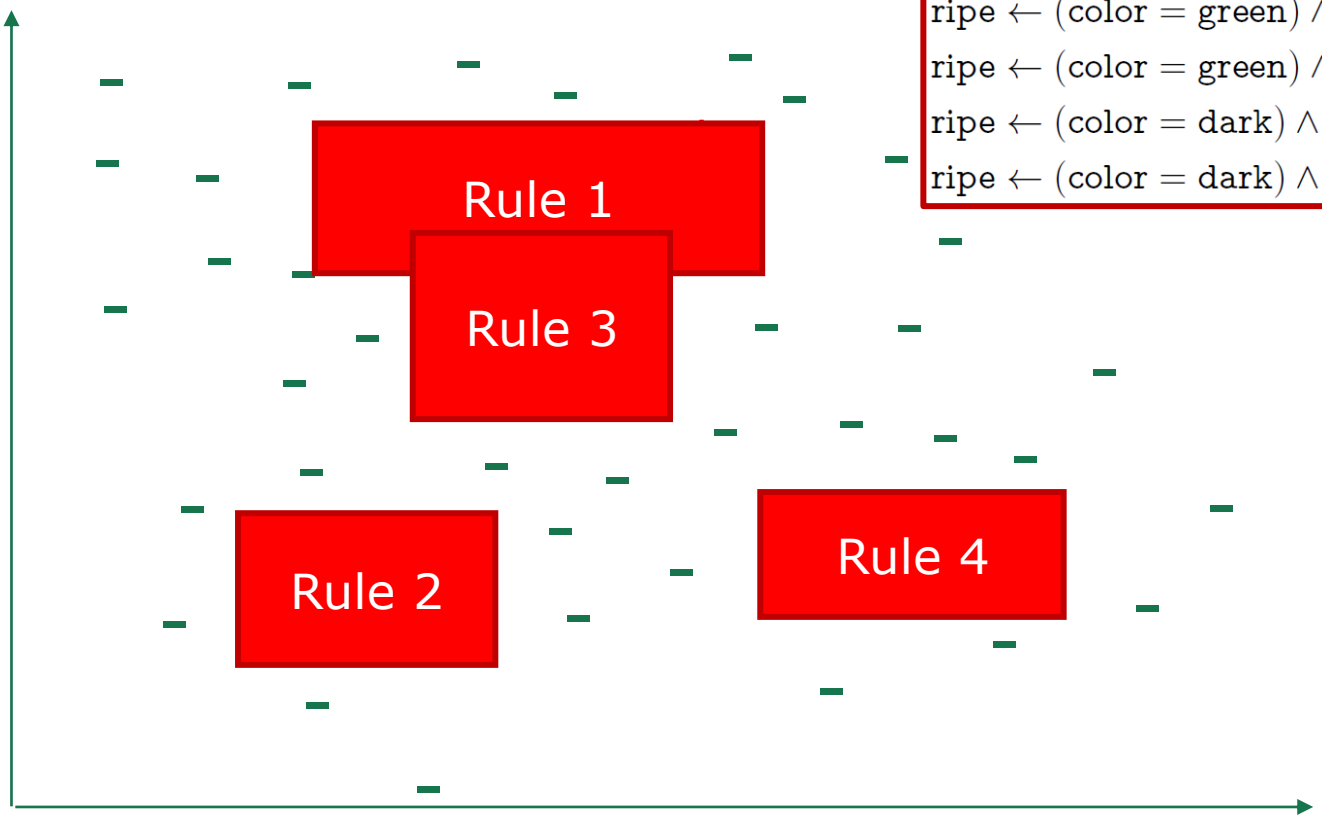
$$\text{ripe} \leftarrow$$

$$\text{ripe} \leftarrow (\text{color} = \text{green}).$$

$$\text{ripe} \leftarrow (\text{color} = \text{green}) \wedge (\text{root} = \text{slightly curly})$$

Single rule learning

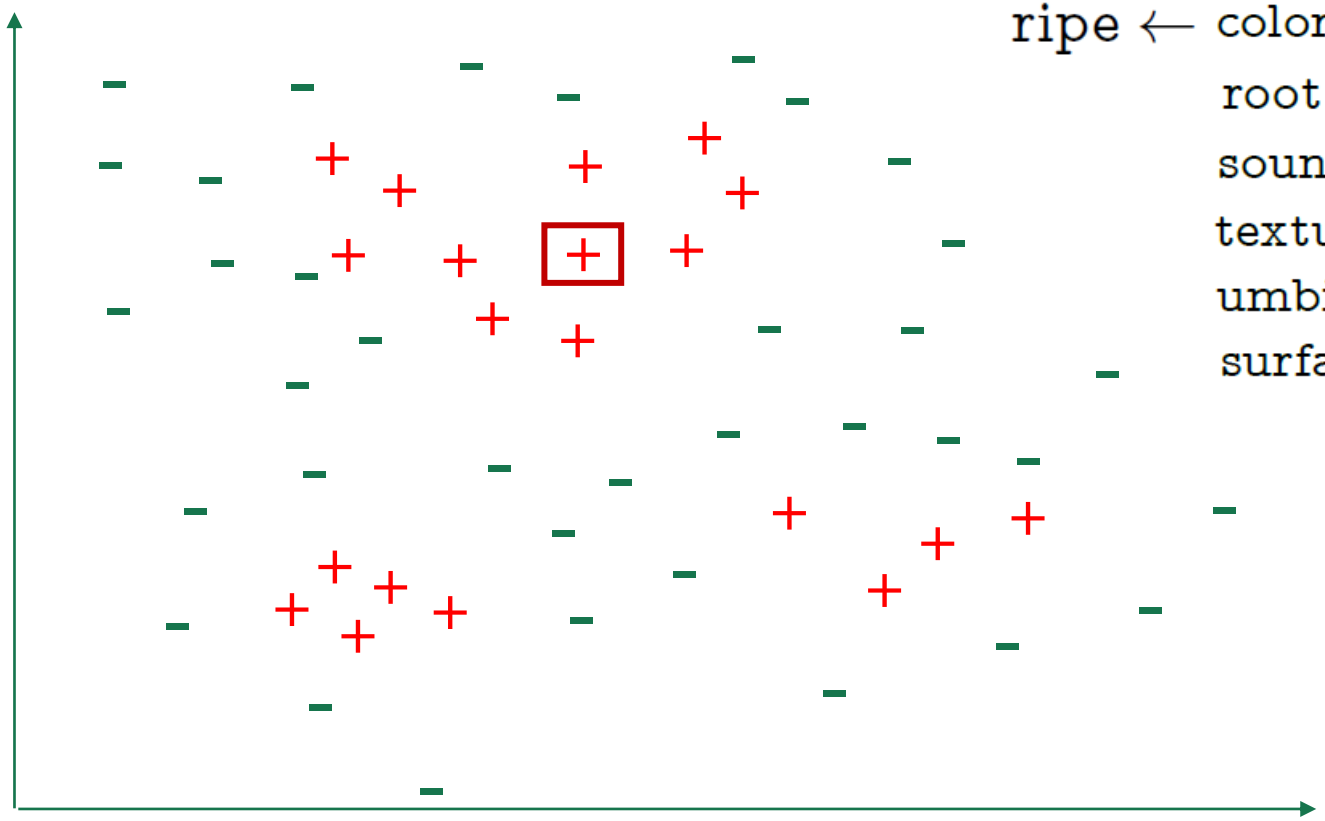
- Top-down strategy: general to special (specialization)



$$\text{ripe} \leftarrow (\text{color} = \text{green}) \wedge (\text{root} = \text{slightly curly});$$
$$\text{ripe} \leftarrow (\text{color} = \text{green}) \wedge (\text{sound} = \text{muffled});$$
$$\text{ripe} \leftarrow (\text{color} = \text{dark}) \wedge (\text{root} = \text{curly});$$
$$\text{ripe} \leftarrow (\text{color} = \text{dark}) \wedge (\text{texture} = \text{slightly blurry})$$

Single rule learning

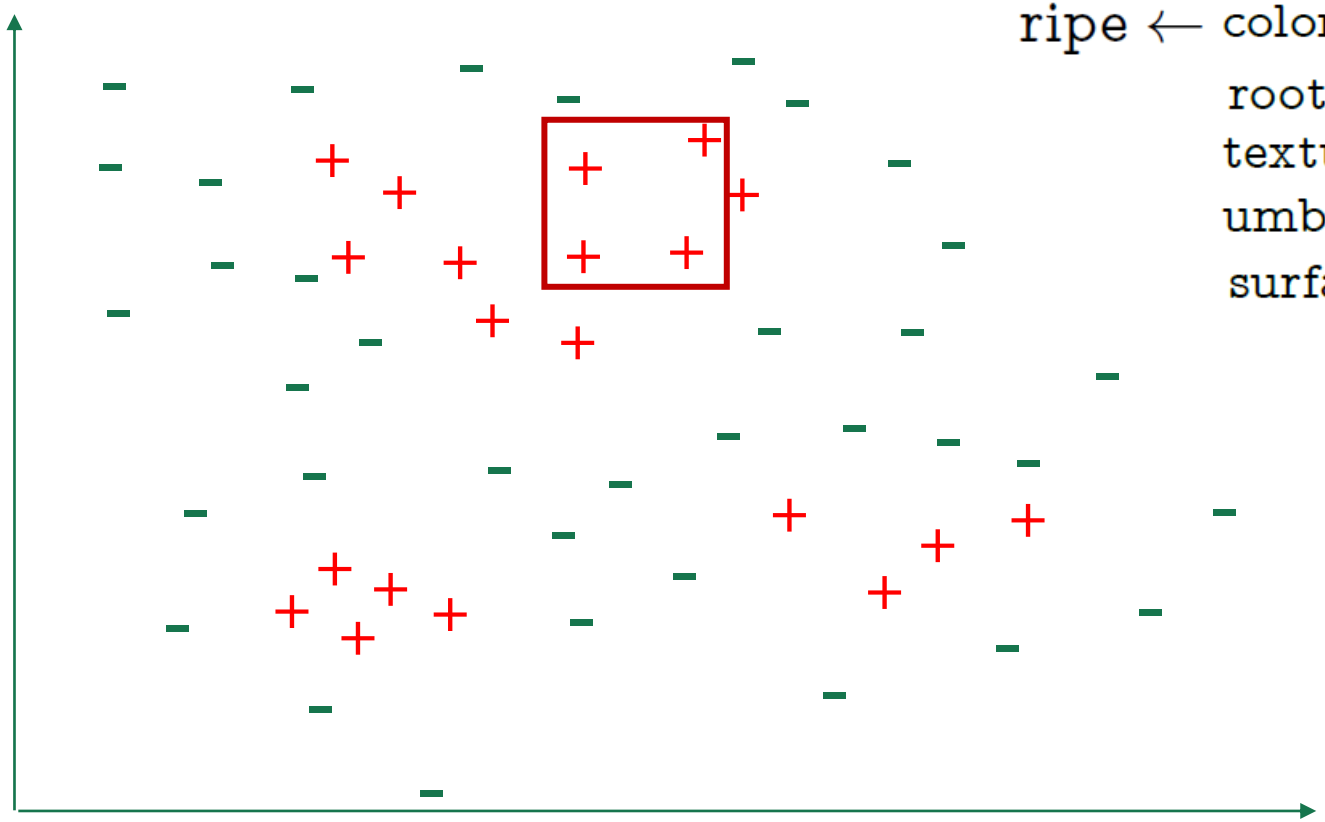
- Bottom-up strategy: special to general (generalization)



ripe ← color = green ∧
root = curly ∧
sound = dull ∧
texture = clear ∧
umbilicus = hollow ∧
surface = hard

Single rule learning

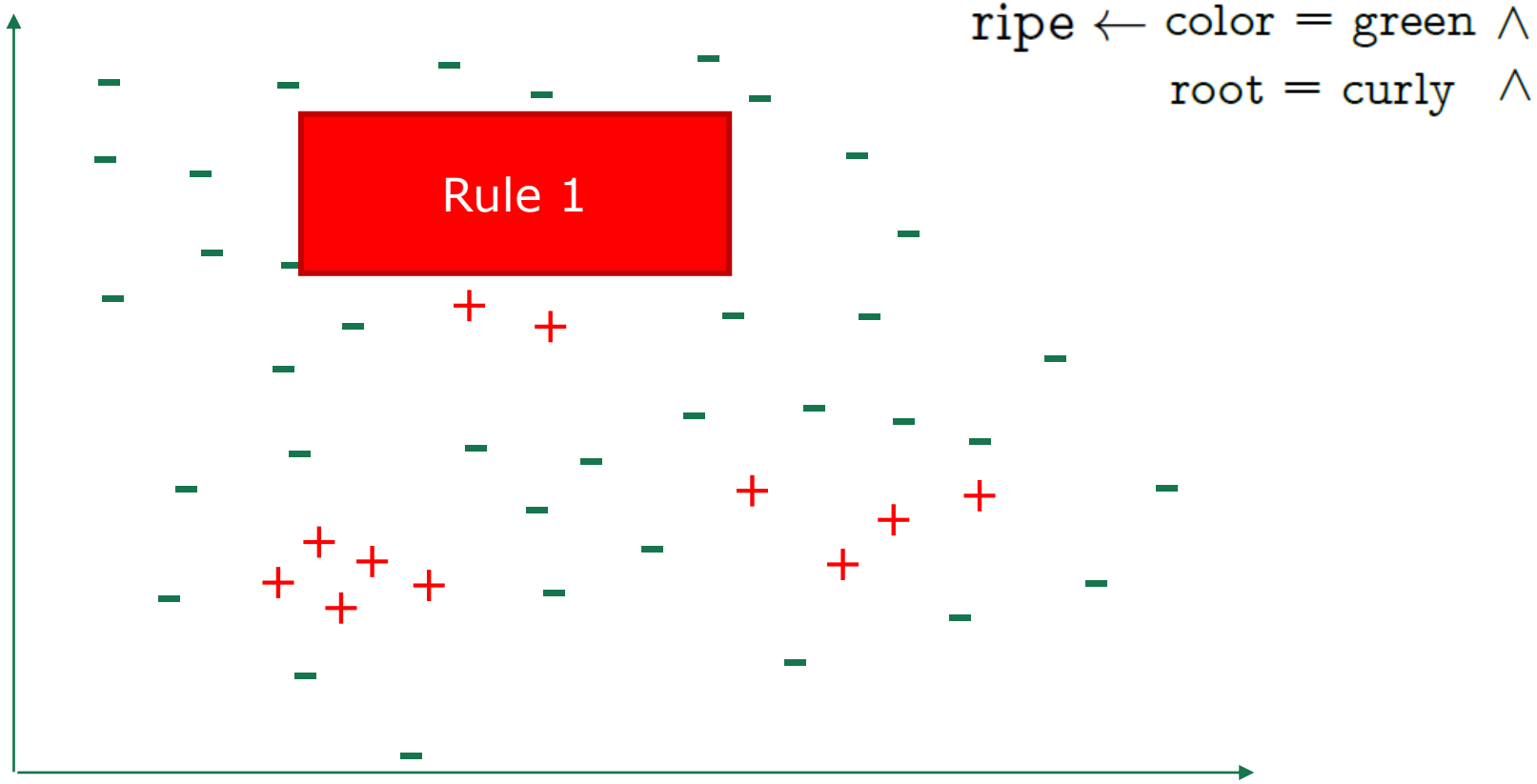
- Bottom-up strategy: special to general (generalization)



ripe ← color = green ∧
root = curly ∧
texture = clear ∧
umbilicus = hollow ∧
surface = hard

Single rule learning

- Bottom-up strategy: special to general (generalization)



Single rule learning

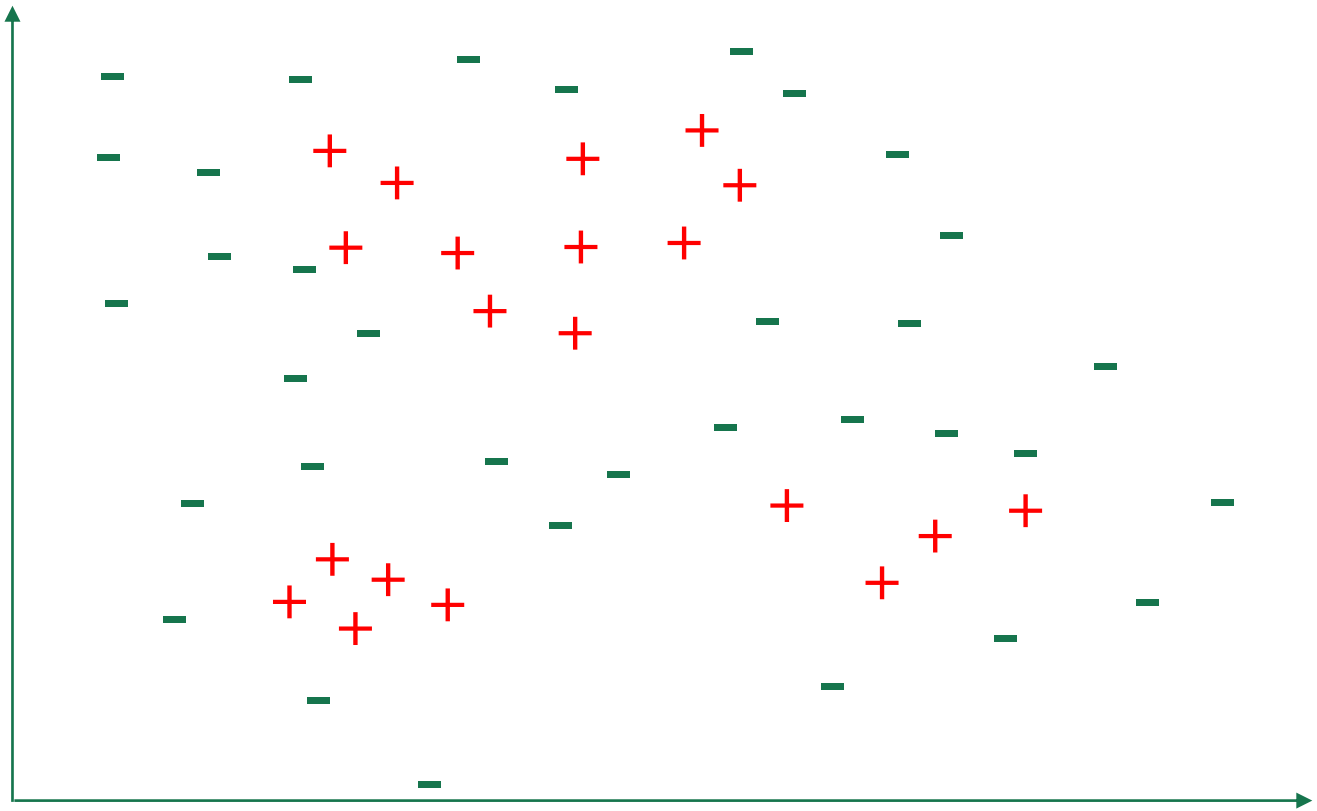
- Rule evaluation: which candidate literal to add/delete
  - Accuracy
  - Information entropy gain (ratio)
  - Gini index
  - ……
- Avoid local optimum
  - Beam search: add the best b literals in the current round to the candidate literal set
  - ……

Outline

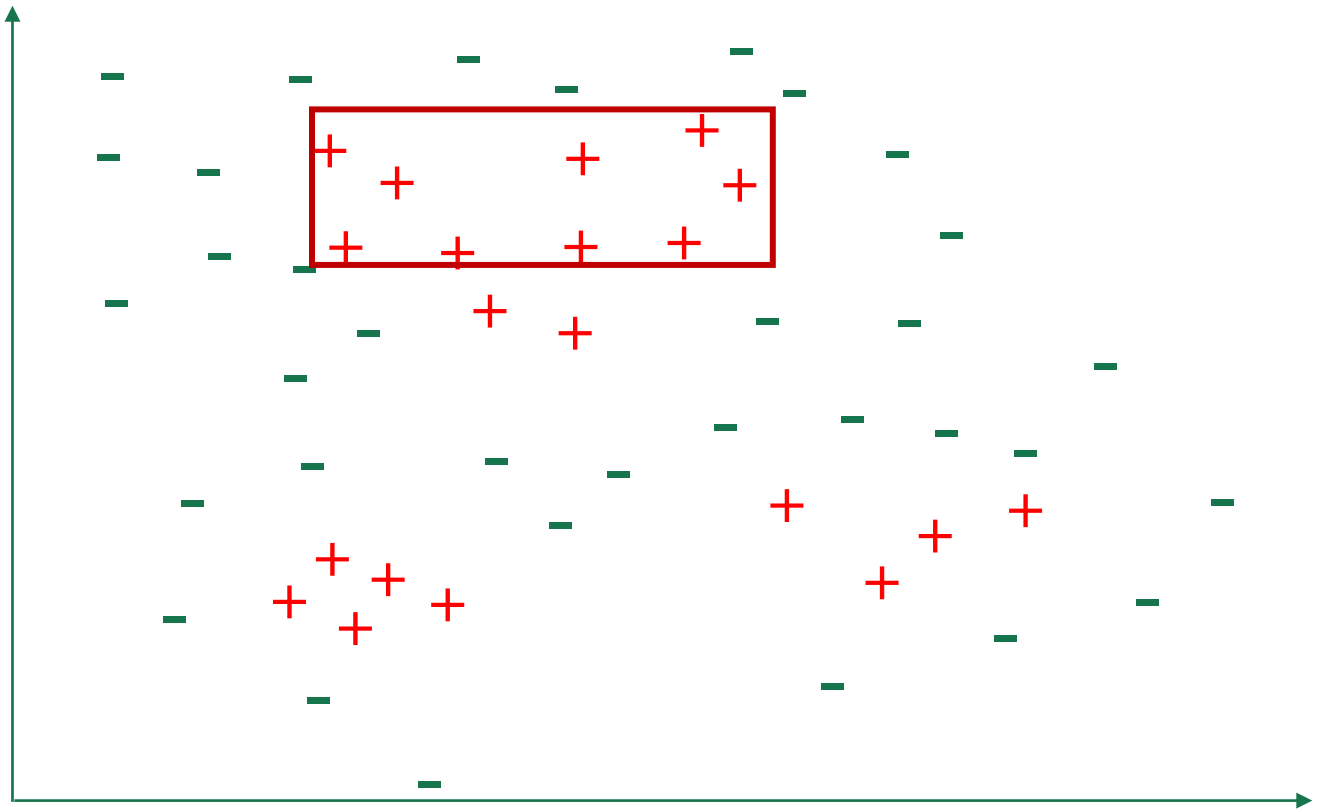
- Basic Concepts
- Sequential Covering
- Pruning Optimization
- First-order Rule Learning
- Inductive Logic Programming

Sequential Covering (Non-optimal)

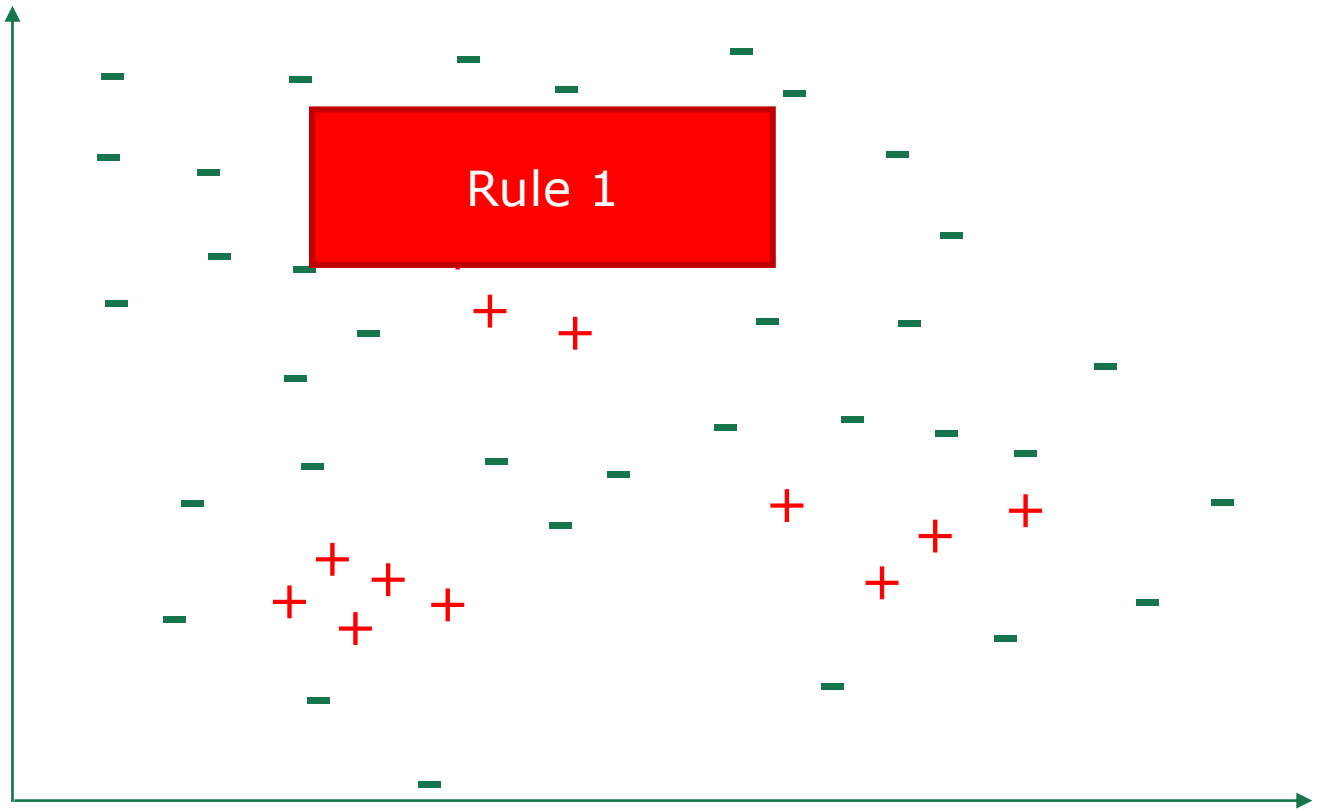
- Non-optimal situation caused by greedy algorithm



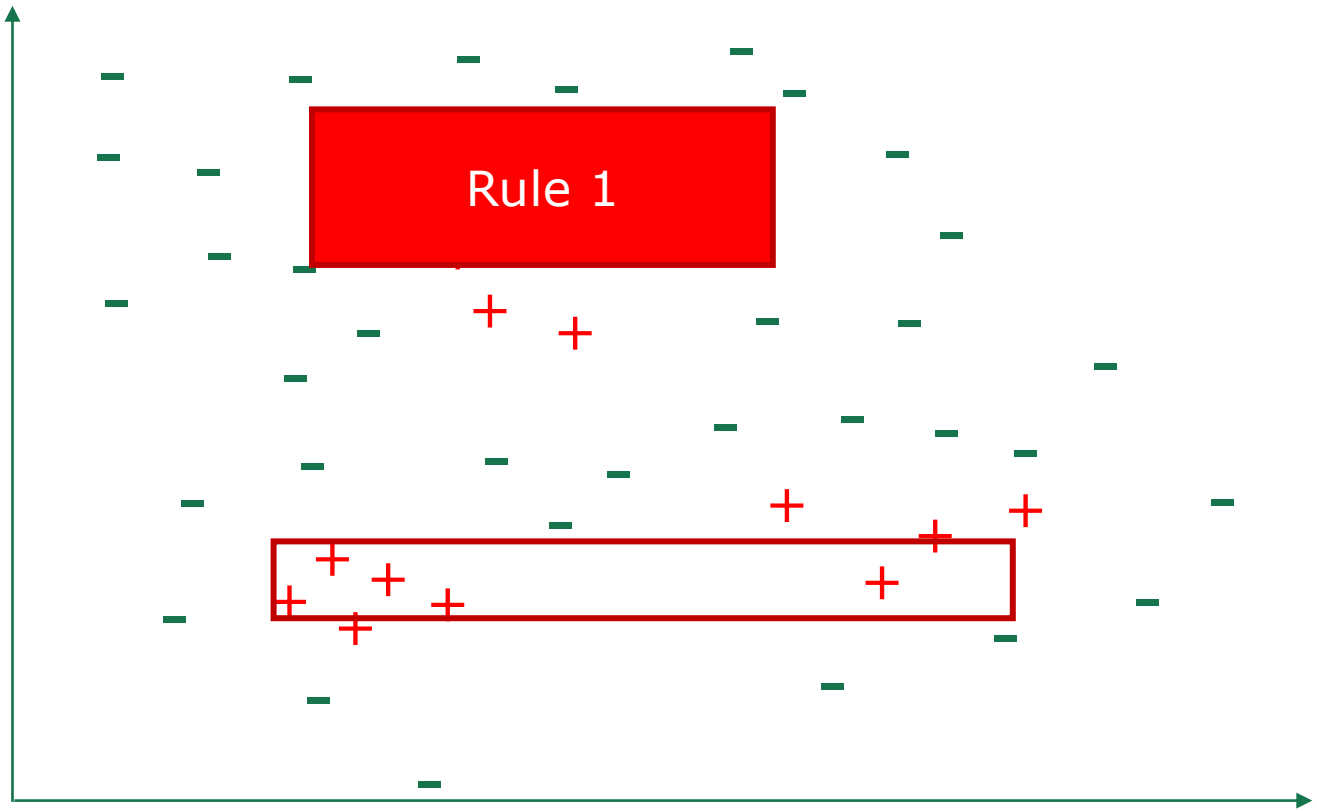
Sequential Covering (Non-optimal)



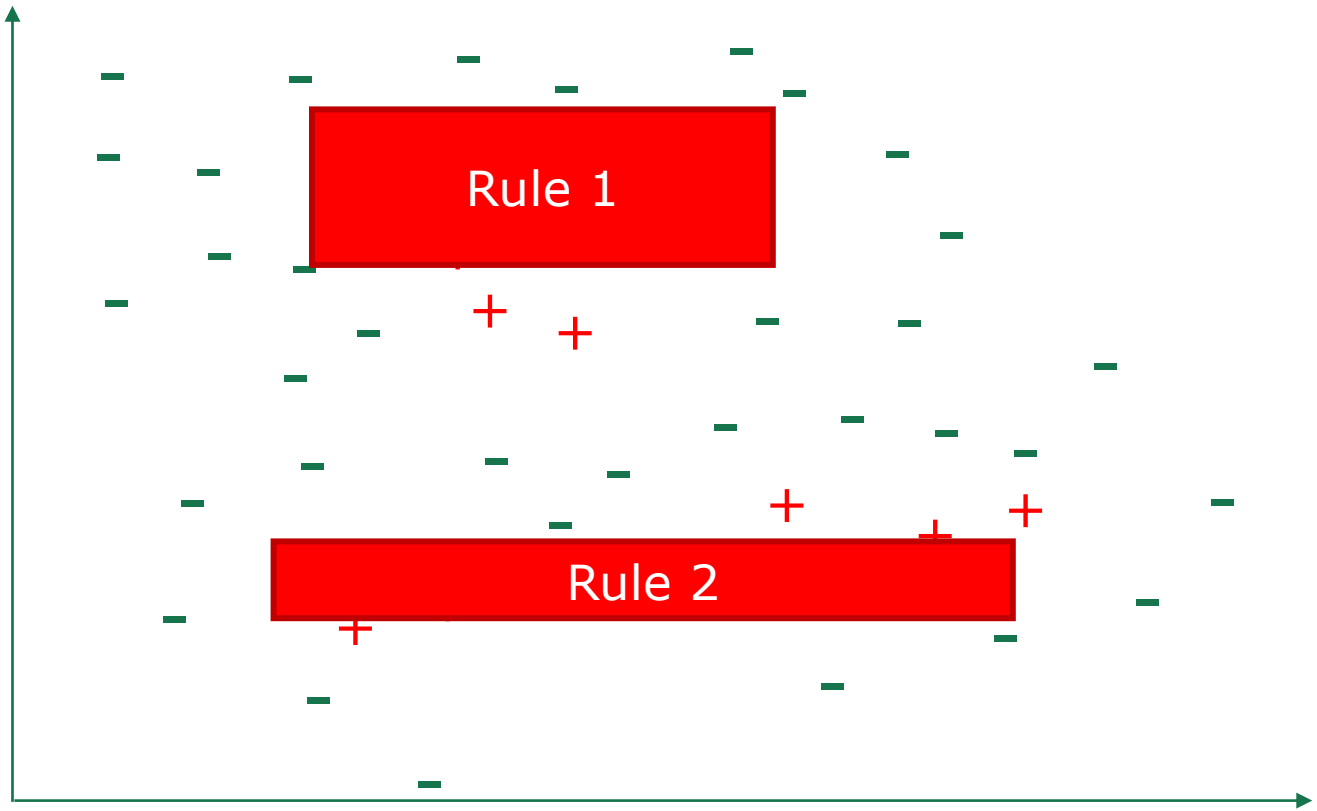
Sequential Covering (Non-optimal)



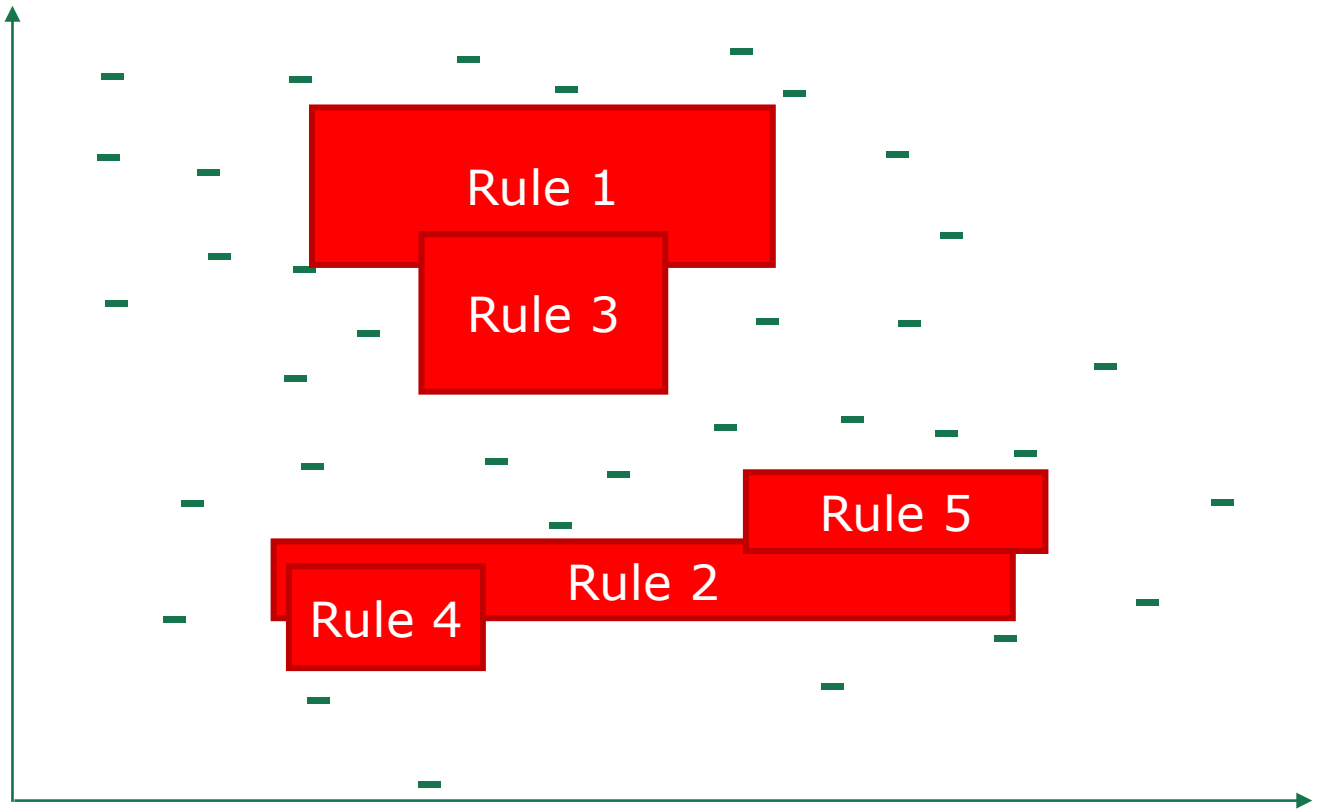
Sequential Covering (Non-optimal)



Sequential Covering (Non-optimal)



Sequential Covering (Non-optimal)



Pruning Optimization

- Pre-pruning
  - **Likelihood Ratio Statistics** (LRS) [Clark and Niblett, 1989]

Empirical probability of positive examples covered by the rule

$$LRS = 2 \cdot (\hat{m}_+ \log_2 \frac{\frac{\hat{m}_+}{\hat{m}_+ + \hat{m}_-}}{\frac{m_+}{m_+ + m_-}} + \hat{m}_- \log_2 \frac{\frac{\hat{m}_-}{\hat{m}_+ + \hat{m}_-}}{\frac{m_-}{m_+ + m_-}})$$

Empirical probability of positive examples

- Post-pruning
  - Reduced Error Pruning (REP) [Brunk and Pazzani, 1991]
    - Evaluate all possible pruning operations (deleting a literal, deleting the entire rule), high complexity (O(m^4))
    - Repeat until no longer improve the performance on validation set
- Combining both
  - IREP[Fürnkranz and Widmer, 1994]
    - Every time a rule r is generated, immediately pruned by REP (reduced complexity: O($m \cdot log^2 m$))
  - IREP*[Cohen, 1995]
    - Improvement of IREP: evaluates accuracy using $\frac{\hat{m}_+ + (m_- - \hat{m}_-)}{m_+ + m_-}$
  - RIPPER[Cohen, 1995]

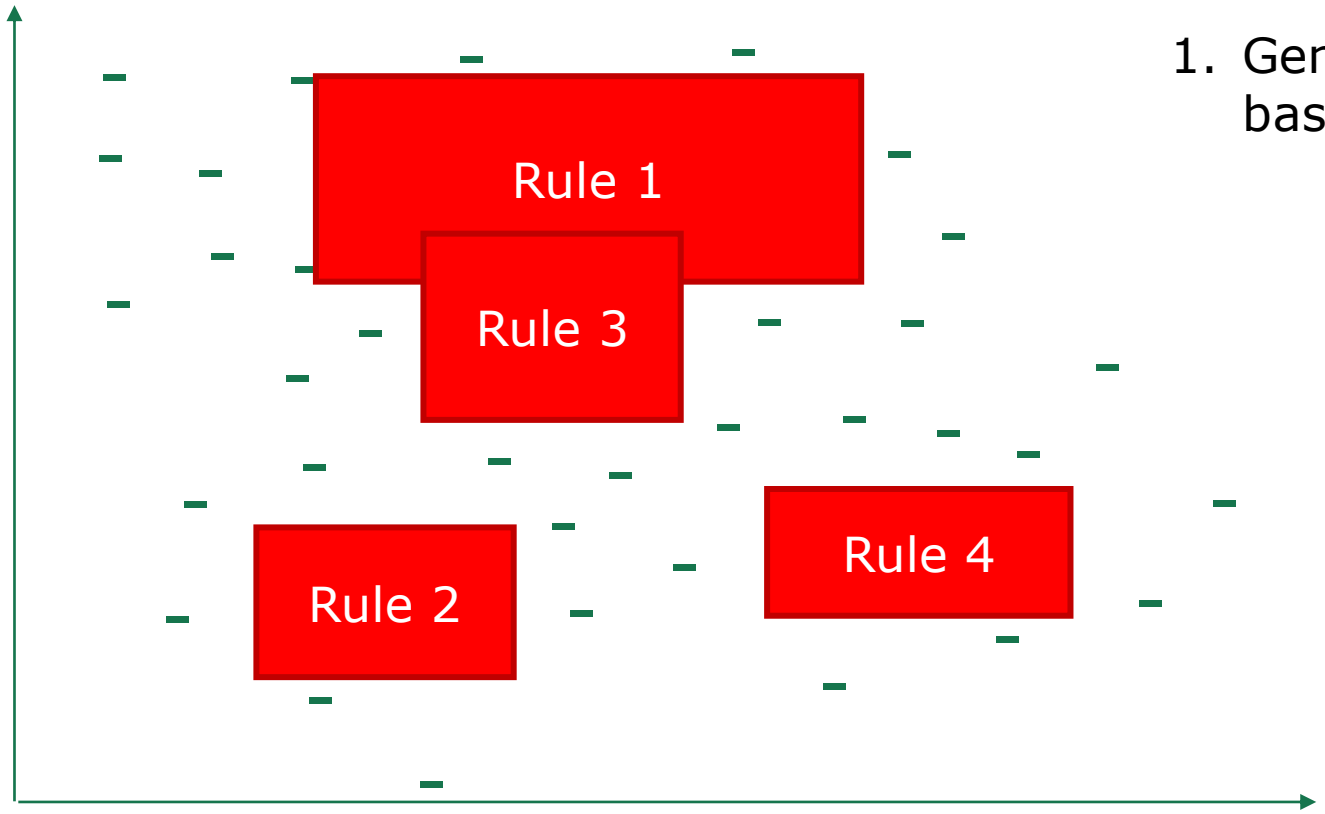
Outline

- Basic Concepts
- Sequential Covering
- Pruning Optimization
  - RIPPER
- First-order Rule Learning
- Inductive Logic Programming

RIPPER (Repeated Incremental Pruning to Produce Error Reduction)

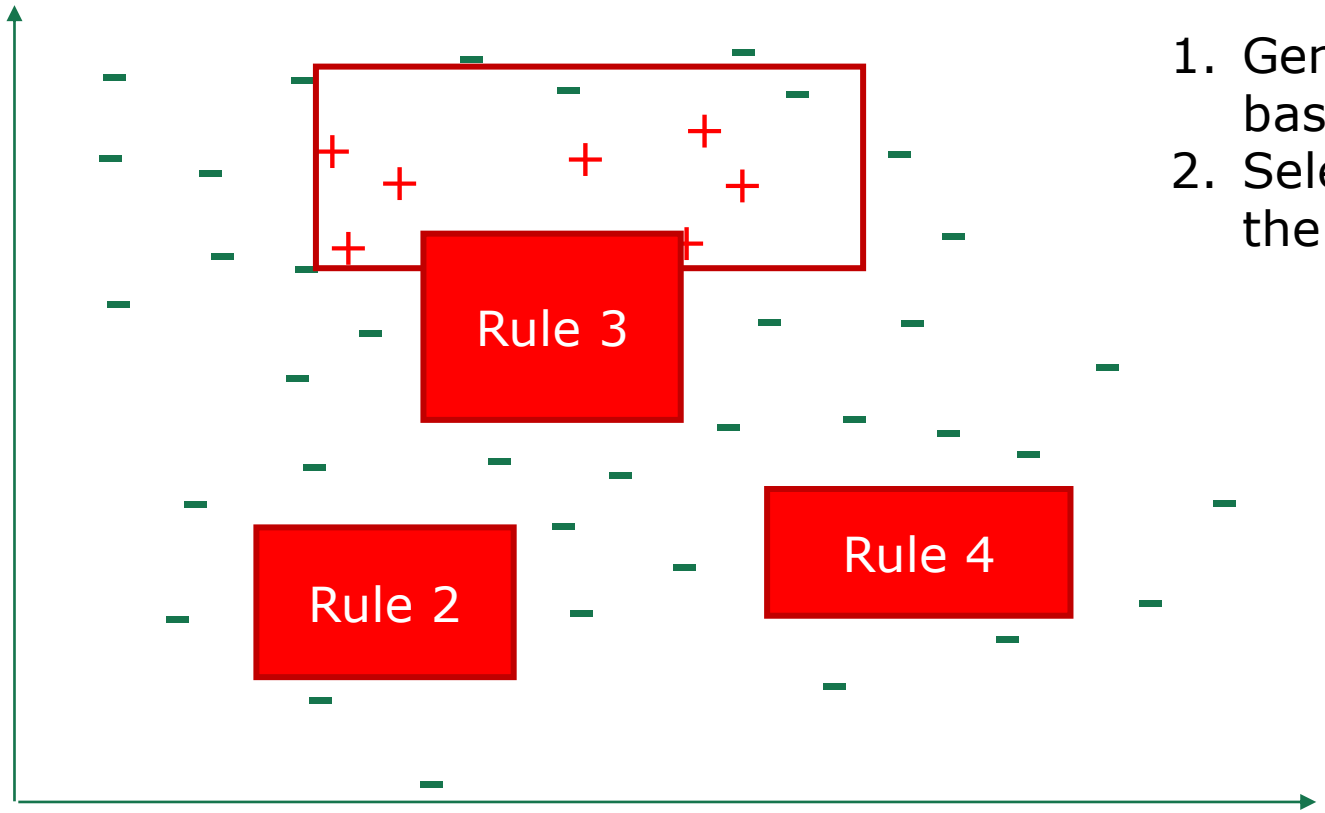
[Cohen, 1995]

Cover two negative samples!



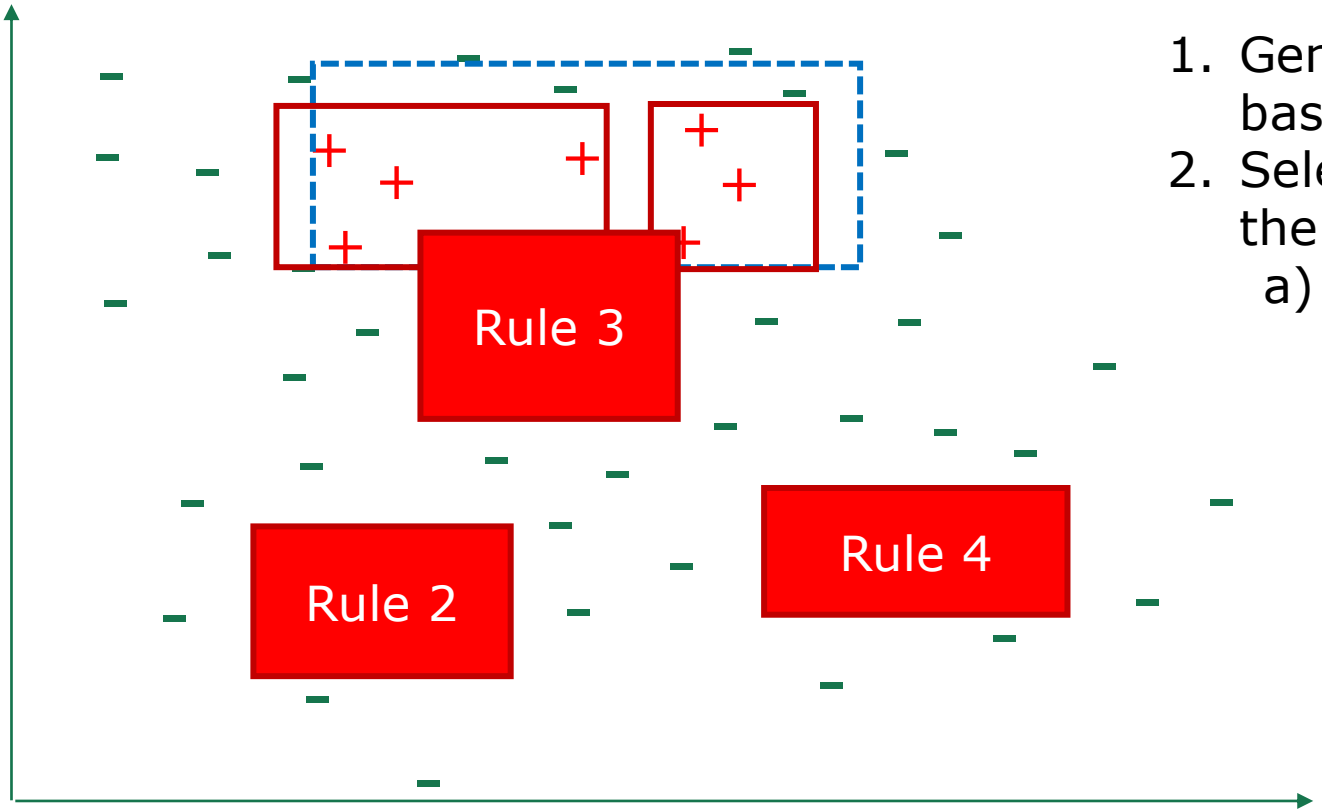
1. Generate rule set based on IREP*

RIPPER [Cohen, 1995]



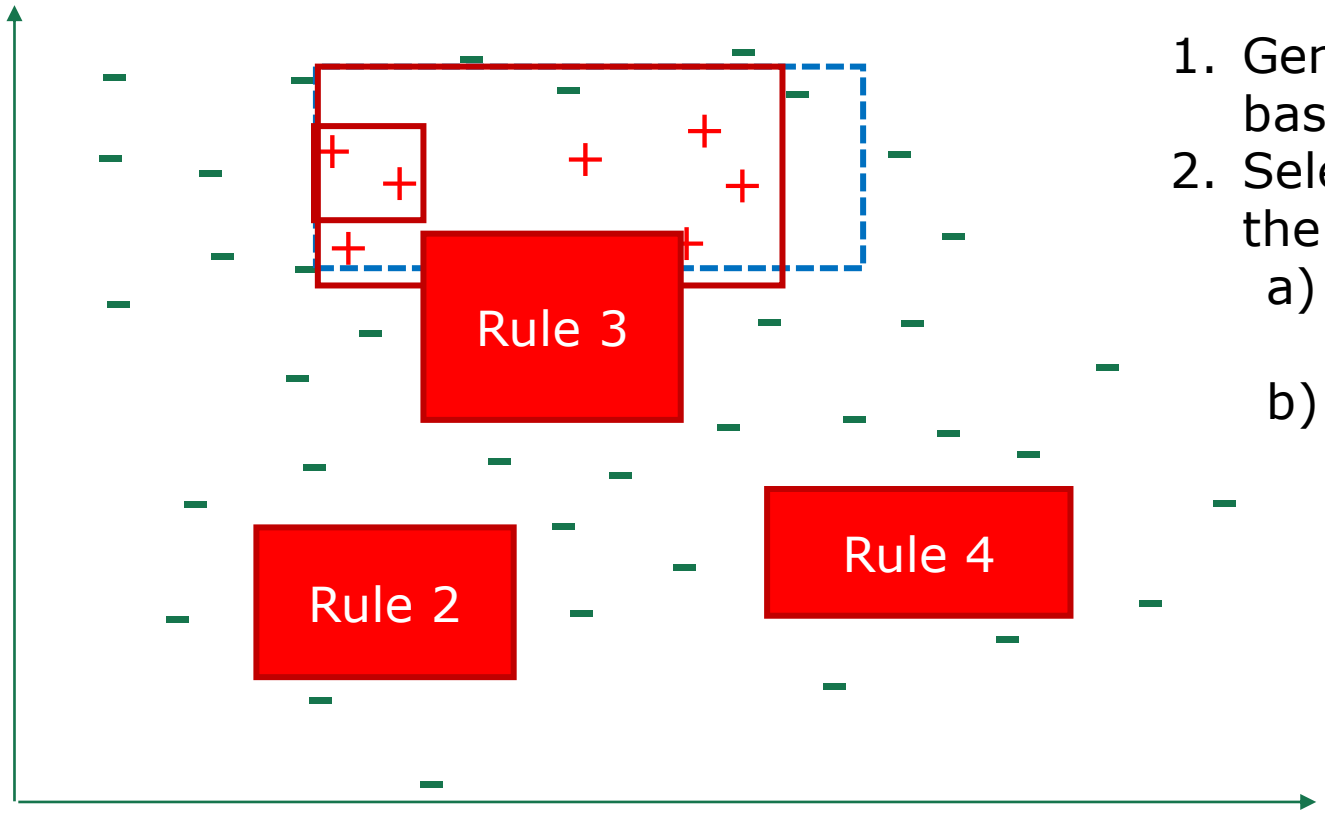
1. Generate rule set based on IREP*
2. Select a rule, find the samples it covers

RIPPER [Cohen, 1995]



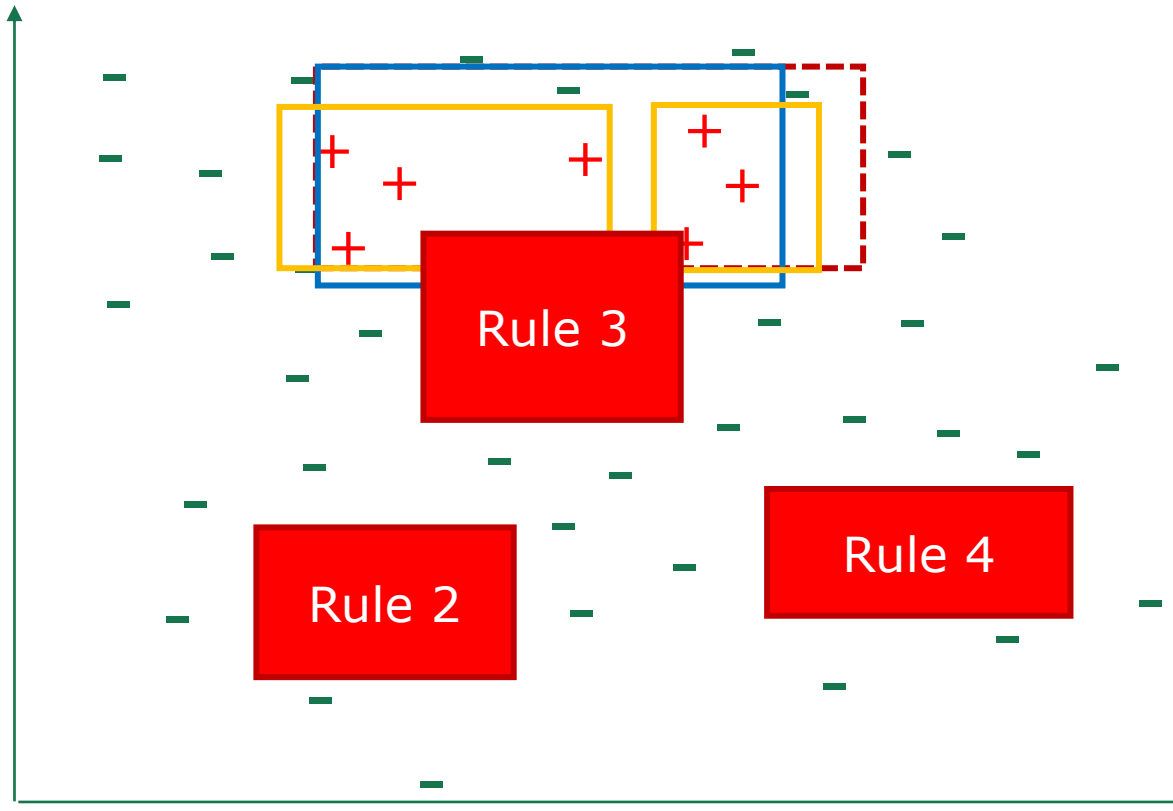
1. Generate rule set based on IREP*
2. Select a rule, find the samples it covers
   a) generate a replacement rule

RIPPER [Cohen, 1995]



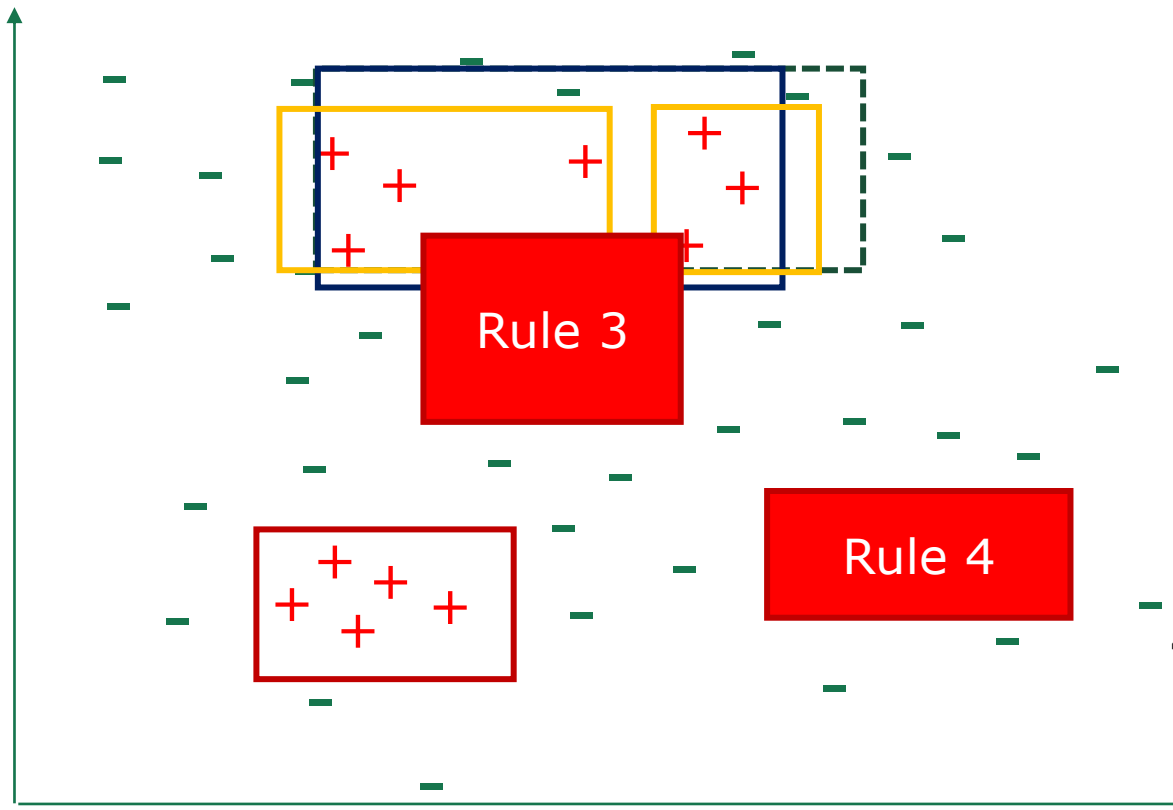
1. Generate rule set based on IREP*
2. Select a rule, find the samples it covers
   a) generate a replacement rule
   b) Specialize the original rule and generalize it

RIPPER [Cohen, 1995]



1. Generate rule set based on IREP*
2. Select a rule, find the samples it covers
   a) generate a replacement rule
   b) specialize the original rule and generalize it
3. Compare the rule sets, and keep the optimal one

RIPPER [Cohen, 1995]



1. Generate rule set based on IREP*
2. Select a rule, find the samples it covers
   a) generate a replacement rule
   b) specialize the original rule and generalize it
3. Compare the rule sets, and keep the optimal one
4. Repeat until no improvement

RIPPER alleviates the locality problem of greedy approach by revisiting rule sets at the end to achieve a better performance.

Outline

- Basic Concepts
- Sequential Covering
- Pruning Optimization
- First-order Rule Learning
- Inductive Logic Programming

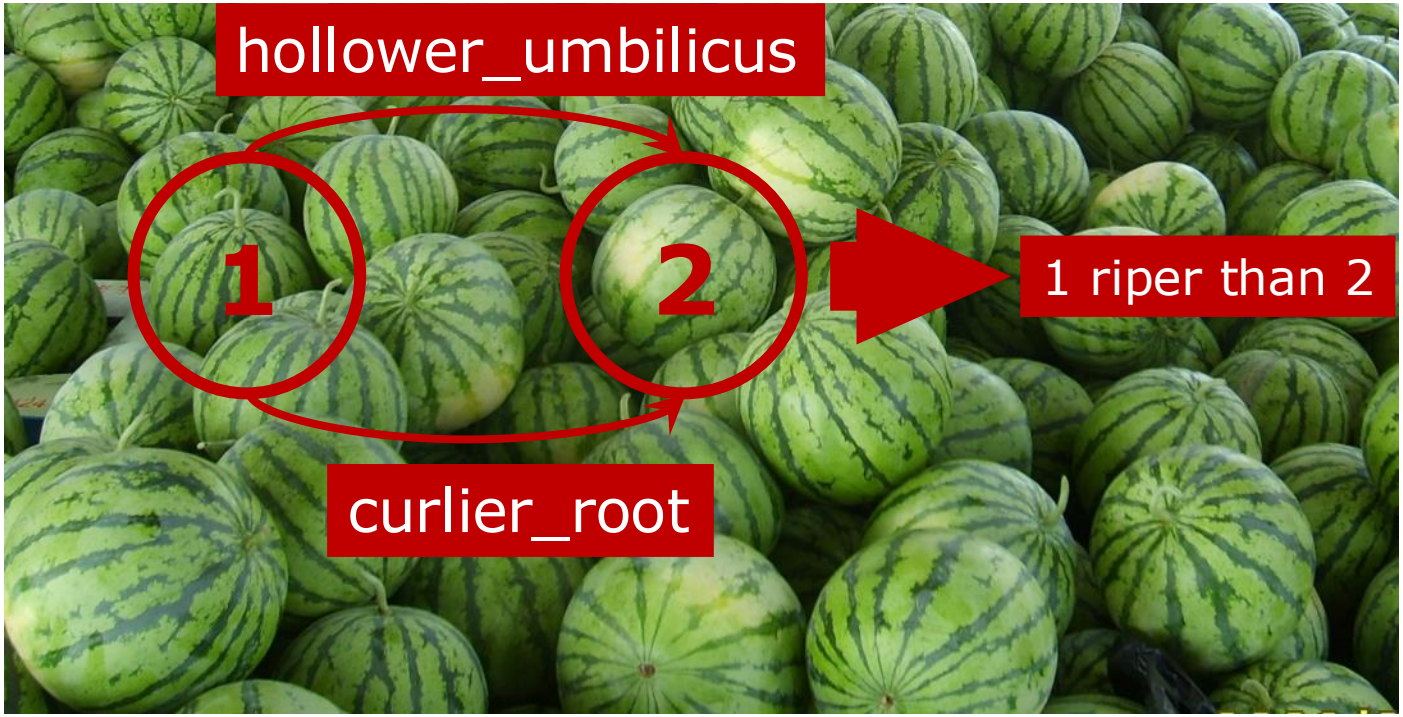
First-order Rule Learning

- Aim of *First-order*: describe the relations between samples

It can be difficult to describe all watermelons with precise feature values

First-order Rule Learning

- Use first-order relation to select *riper* watermelon



Omit the quantifiers when the context is clear

$$(\forall X, \forall Y)(\text{riper}(X, Y) \leftarrow \text{curlier\_root}(X, Y) \wedge \text{hollower\_umbilicus}(X, Y)).$$

First-order Rule Learning

Attribute-value data
[Propositional logic]

ID	color	root	sound	texture	umbilicus	surface	ripe
1	green	curly	muffled	clear	hollow	hard	true
2	dark	curly	dull	clear	hollow	hard	true
3	dark	curly	muffled	clear	hollow	hard	true
6	green	slightly curly	muffled	clear	slightly hollow	soft	true
7	dark	slightly curly	muffled	slightly blurry	slightly hollow	soft	true
10	green	straight	crisp	clear	flat	soft	false
14	light	slightly curly	dull	slightly blurry	hollow	hard	false
15	dark	slightly curly	muffled	clear	slightly hollow	soft	false
16	light	curly	muffled	blurry	flat	hard	false
17	green	curly	dull	slightly blurry	slightly hollow	hard	false

- darkness of color: dark > green > light;
- curliness of root: curly > slightly curly > straight;
- dullness of sound: dull > muffled > crisp;
- clearness of texture: clear > slightly blurry > blurry;
- hollowness of umbilicus: hollow > slightly hollow > flat;
- hardness of surface: hard > soft.

Background knowledge

Relational data
[First-order logic]

hollower_umbilicus(1, 15)	hollower_umbilicus(15, 10)	hollower_umbilicus(15, 16)
hollower_umbilicus(17, 10)	hollower_umbilicus(17, 16)	harder_surface(1, 6)
harder_surface(1, 7)	harder_surface(1, 10)	harder_surface(1, 15)
...	...	...
harder_surface(17, 6)	harder_surface(17, 7)	harder_surface(17, 10)
harder_surface(17, 15)		
riper(1, 10)	riper(1, 14)	riper(1, 15)
...	...	...
riper(1, 16)	riper(7, 14)	riper(7, 15)
riper(7, 16)	riper(7, 17)	¬riper(10, 1)

examples

FOIL[Quinlan, 1990]

- Sequential covering framework
- Top-down inductive strategy
  - Combinations of all possible predicates and variables as candidate literals

$$\text{riper}(X,Y) \leftarrow \begin{cases} \text{darker\_color}(X,Y), & \text{darker\_color}(Y,X), \\ \text{darker\_color}(X,Z), & \text{darker\_color}(Z,X), \\ \text{darker\_color}(Y,Z), & \text{darker\_color}(Z,Y), \\ \text{darker\_color}(X,X), & \text{darker\_color}(Y,Y), \\ \text{curlier\_root}(X,Y), & \text{duller\_sound}(X,Y), \\ \ldots & \ldots \end{cases}$$

  - Select literals by evaluating the *FOIL gain*

$$\text{F\_GAIN} = \hat{m}_+ (\log_2 \frac{\hat{m}_+}{\hat{m}_+ + \hat{m}_-} - \log_2 \frac{m_+}{m_+ + m_-})$$

- Post-prune the rule set

Outline

- Basic Concepts
- Sequential Covering
- Pruning Optimization
- First-order Rule Learning
- Nints to Inductive Logic Programming (ILP)

Inductive Logic Programming (ILP)[Muggleton, 1991]

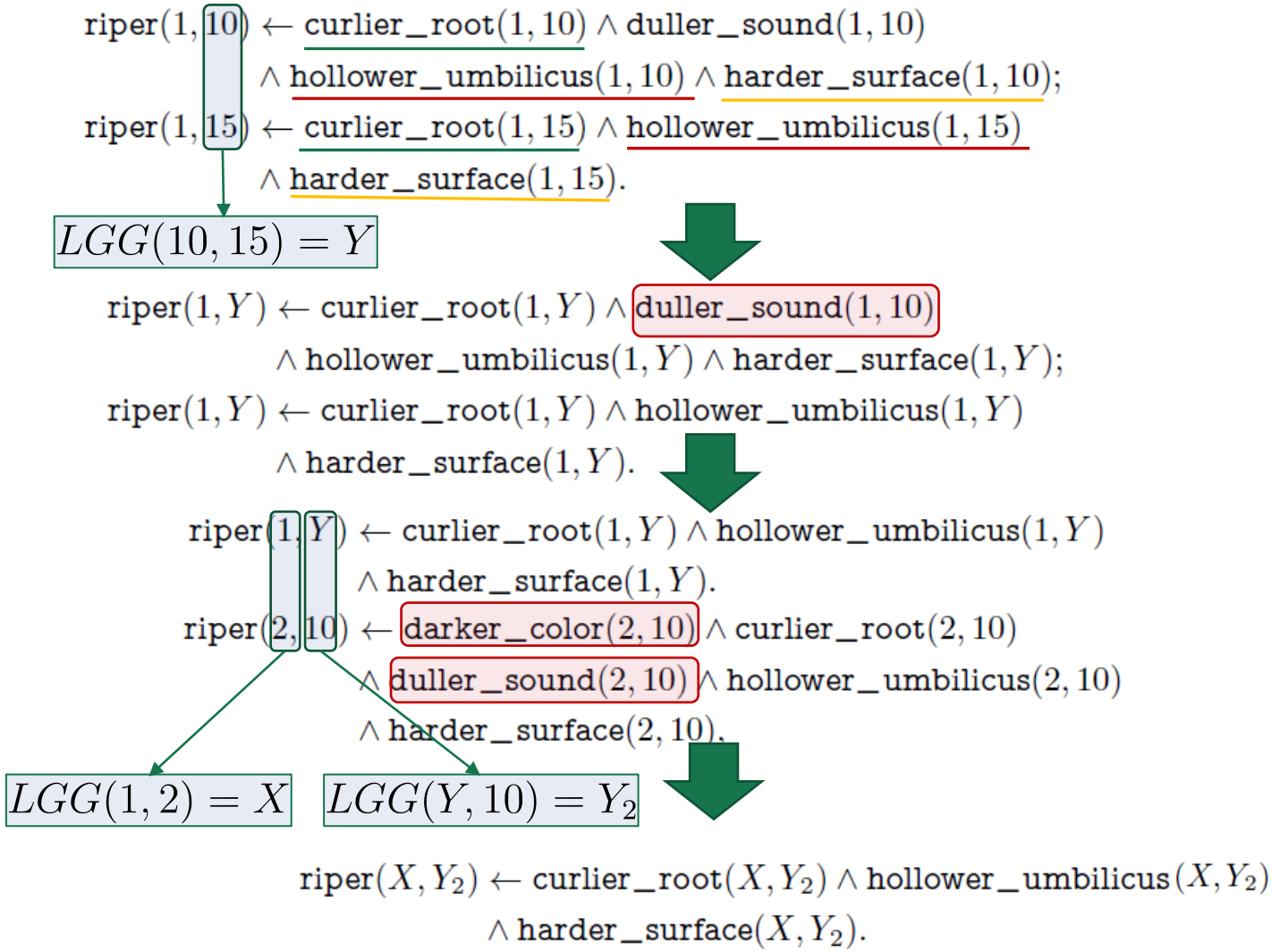
- Target: Completely learn first-order rules (Horn clauses)
- Still use sequential covering to learn rule set
- Bottom-up strategy to learn single rule generally
  - No need to enumerate all possible candidate rules
  - The search for the target concept is maintained in a local area near the sample
  - The search space of the top-down strategy increases exponentially with the rule length

Least General Generalization (LGG)[Plotkin, 1970]

- Generalization: transform a rule with low coverage into a rule with high coverage
- General: coverage as high as possible
- Least: changes to the original rules as small as possible

- Steps to find LGG of two rules:
  - Find all literals that have the same predicate in two rules
  - Check each pair of constants at the same position in literals:
    - $LGG(t,t) = t$
    - $LGG(s,t), s \neq t$, $LGG(s,t) = V$, the constants are replaced by a new variable
- Delete all literals that do not have a common predicate

Least General Generalization (LGG) [Plotkin, 1970]



Least General Generalization (LGG) [Plotkin, 1970]

- Other ILP algorithm based on LGG
  - Consider negative literals
  - Different initialization options
    - Multiple special rules
    - Consider all background knowledge (RLGG) [Plotkin, 1971]:

      Relative LGG (RLGG) takes background knowledge into consideration by setting the bottom rule of sample e as $e \leftarrow K$, where K is the conjunction of all atoms in background knowledge.

Conclusion

- Logic rules
  - Propositional rules, first-order rules
- Rule set learning: sequential covering
  - Single rule learning: top-down, bottom-up
  - Pruning and post-pruning
- First-order Rule Learning
- Hints to Inductive Logic Programming
  - Least General Generalization (LGG)

Tools

- JRIP of WEKA (http://weka.sourceforge.net/doc.dev/weka/classifiers/rules/JRip.html)
- Integrated tool of ILP--ALEPH (http://www.cs.ox.ac.uk/activities/machlearn/Aleph/aleph.html)
- Relative Least General Generalization: GOLEM (http://www.doc.ic.ac.uk/~shm/golem.html)
- Inverse Entailment: Progol (http://www.doc.ic.ac.uk/~shm/progol.html)
- Open source Prolog environment:
  - YAP (http://www.dcc.fc.up.pt/~vsc/Yap/)
  - SWI-Prolog (http://www.swi-prolog.org/)

Thanks!